CPT102 Algorithm Five-Question Review

期末大题模板：similar algorithm / complexity / stability / recursive version / recursive disadvantage

How to use this PDF: For each algorithm, memorize the short principle, the Big-O reason, and the recursive-vs-iterative tradeoff. For non-sorting algorithms, "stable" is normally not applicable; write that clearly in exams.

Source coverage

Covered from uploaded CPT102 lectures: Weeks 4-6 searching/sorting; Lecture 8a Union-Find; Lecture 8b Priority Queue and Binary Heap; Lectures 9-11 Graphs, Shortest Paths, MST, SCC; Lecture 12 BST / balanced trees; Lecture 13 Hashing.

One-page comparison table

Algorithm	Type	Main time	Stable?	Recursive note
Linear Search	Search	O(n)	No	Recursive simple, worse space
Binary Search	Search	O(log n)	No	Natural recursive
Bubble Sort	Sort	O(n^2)	Yes, if only swap >	Recursive prefix possible
Selection Sort	Sort	O(n^2)	Usually no	Recursive suffix possible
Insertion Sort	Sort	O(n^2), best O(n)	Yes	Recursive prefix possible
Merge Sort	Sort	O(n log n)	Yes if merge left-first	Naturally recursive
Quick Sort	Sort	Avg O(n log n), worst O(n^2)	No	Naturally recursive
Counting Sort	Sort	O(n+k)	Yes if cumulative stable	Loop-based
Radix Sort	Sort	O(d(n+k))	Yes if stable pass	Loop-based over digits
Union-Find	Connectivity	depends variant	N/A	find can be recursive
Binary Heap PQ	Priority Queue	add/delMin O(log n)	N/A	swim/sink can recurse
DFS/BFS	Graph traversal	O(V+E)	N/A	DFS recursive, BFS queue
Dijkstra	Shortest path	O((V+E)logV)	N/A	PQ loop preferred
Prim/Kruskal	MST	O(ElogV)/O(ElogE)	N/A	Loop preferred
BST get/put	Map	O(h), worst O(n)	N/A	Natural recursive
Hash Table	Set/Map	avg O(1), worst O(n)	N/A	Loop preferred

Algorithm cards

0. WeirdSort model answer

Exam-style template from the screenshot.

Q1 Similar to what? / Principle	Most similar to Selection Sort. It scans all later positions j for each i and swaps whenever it finds a smaller element. Unlike standard selection sort, it swaps immediately rather than remembering the minimum index and swapping once.
Q2 Time complexity	O(n^2). The outer loop runs about n times; the inner loop runs about n-i times. Total comparisons are (n-1)+(n-2)+...+1 = O(n^2).
Q3 Stable?	Not stable. Example: [2a, 2b, 1]. When i=0 and j=2, 1 is swapped with 2a, giving [1, 2b, 2a]; equal keys 2a and 2b changed order.
Q4 Recursive vs iterative	The iterative version uses two nested loops. A recursive version can replace the two loops by recursion over i and recursion over j.
Q5 Recursive disadvantage	Same O(n^2) time, but recursive version uses extra call stack space and is harder to read. If implemented with one recursive call per loop step, stack depth can be O(n) or even O(n^2) depending on style.

Iterative pseudocode

```
Procedure WeirdSort(array)
  n <- array.length
  for i <- 0 to n - 2 do
    for j <- i + 1 to n - 1 do
      if array[j] < array[i] then
        swap(array[j], array[i])
```

Recursive pseudocode

```
Procedure WeirdSort(array)
  WeirdOuter(array, 0)

Procedure WeirdOuter(array, i)
  if i >= array.length - 1 then return
  WeirdInner(array, i, i + 1)
  WeirdOuter(array, i + 1)

Procedure WeirdInner(array, i, j)
  if j >= array.length then return
  if array[j] < array[i] then
    swap(array[j], array[i])
  WeirdInner(array, i, j + 1)
```

1. Linear Search

Weeks 4-5: searching; iterative and recursive versions.

Q1 Similar to what? / Principle	Principle: check items one by one from left to right until the target is found or the array ends. Similar to scanning a list manually.
Q2 Time complexity	Worst O(n), best O(1), average O(n). Space: iterative O(1), recursive O(n) due to call stack.
Q3 Stable?	Not a sorting algorithm, so stability is not applicable.

Q4 Recursive vs iterative	Iteration uses a loop over indices/items. Recursion checks one index, then calls itself on the next index.
Q5 Recursive disadvantage	Recursive version has no speed benefit and uses O(n) stack space. It may cause stack overflow on very large arrays.

Iterative pseudocode

```
Procedure LinearSearch(arr, val)
  for i <- 0 to arr.length - 1 do
    if arr[i] = val then return true
  return false
```

Recursive pseudocode

```
Procedure LinearSearch(arr, val)
  return LSHelper(arr, val, 0)

Procedure LSHelper(arr, val, i)
  if i = arr.length then return false
  if arr[i] = val then return true
  return LSHelper(arr, val, i + 1)
```

2. Binary Search

Week 5: sorted arrays; iterative and recursive versions.

Q1 Similar to what? / Principle	Principle: only works on a sorted array. Compare with the middle element and discard half of the search range each step. Similar to divide-and-conquer.
Q2 Time complexity	O(log n) time because the search interval halves each step. Space: iterative O(1), recursive O(log n).
Q3 Stable?	Not a sorting algorithm; stability is not applicable.
Q4 Recursive vs iterative	Iteration maintains low and high. Recursion passes low and high into the next call.
Q5 Recursive disadvantage	Recursive version is elegant, but uses stack frames. Iterative version is usually simpler for avoiding stack overhead.

Iterative pseudocode

```
Procedure BinarySearch(arr, val)
  low <- 0; high <- arr.length - 1
  while low <= high do
    mid <- (low + high) / 2
    if arr[mid] = val then return true
    else if arr[mid] < val then low <- mid + 1
    else high <- mid - 1
  return false
```

Recursive pseudocode

```
Procedure BinarySearch(arr, val)
  return BSHelper(arr, val, 0, arr.length - 1)

Procedure BSHelper(arr, val, low, high)
  if low > high then return false
  mid <- (low + high) / 2
  if arr[mid] = val then return true
  if arr[mid] < val then
    return BSHelper(arr, val, mid + 1, high)
  else
```

```
    return BSHelper(arr, val, low, mid - 1)
```

3. Bubble Sort

Week 5: basic decrease-and-conquer sorting.

Q1 Similar to what? / Principle	Principle: repeatedly compare adjacent items and swap if they are in the wrong order. Large elements bubble to the right end.
Q2 Time complexity	Worst/average O(n^2). Best O(n) if using an early-stop flag on already sorted input; otherwise O(n^2). Space O(1).
Q3 Stable?	Stable if the comparison swaps only when arr[j] > arr[j+1], not when equal. Equal elements keep their relative order.
Q4 Recursive vs iterative	Iteration uses nested loops. Recursion can bubble the max to the end, then recursively sort the prefix of length n-1.
Q5 Recursive disadvantage	Recursive version still performs O(n^2) comparisons and uses O(n) stack space; no practical advantage.

Iterative pseudocode

```
Procedure BubbleSort(arr)
    n <- arr.length
    for end <- n - 1 downto 1 do
        for j <- 0 to end - 1 do
            if arr[j] > arr[j+1] then
                swap(arr[j], arr[j+1])
```

Recursive pseudocode

```
Procedure BubbleSort(arr)
    BubbleSortPrefix(arr, arr.length)

Procedure BubbleSortPrefix(arr, n)
    if n <= 1 then return
    for j <- 0 to n - 2 do
        if arr[j] > arr[j+1] then
            swap(arr[j], arr[j+1])
    BubbleSortPrefix(arr, n - 1)
```

4. Selection Sort

Week 5: basic decrease-and-conquer sorting.

Q1 Similar to what? / Principle	Principle: repeatedly find the smallest item in the unsorted suffix and swap it into the first unsorted position.
Q2 Time complexity	O(n^2) comparisons in best/average/worst cases. Space O(1).
Q3 Stable?	Usually not stable because swapping the minimum into position can move an earlier equal element behind a later equal element.
Q4 Recursive vs iterative	Iteration uses i for the boundary and j to find the minimum. Recursion selects the minimum for position start, then sorts start+1.
Q5 Recursive disadvantage	Recursive version adds O(n) stack space while keeping O(n^2) time.

Iterative pseudocode

```
Procedure SelectionSort(arr)
    for i <- 0 to arr.length - 2 do
```

```
    min <- i
    for j <- i + 1 to arr.length - 1 do
      if arr[j] < arr[min] then min <- j
    swap(arr[i], arr[min])
```

Recursive pseudocode

```
Procedure SelectionSort(arr)
  SelectSortFrom(arr, 0)

Procedure SelectSortFrom(arr, start)
  if start >= arr.length - 1 then return
  min <- start
  for j <- start + 1 to arr.length - 1 do
    if arr[j] < arr[min] then min <- j
  swap(arr[start], arr[min])
  SelectSortFrom(arr, start + 1)
```

5. Insertion Sort

Week 5: basic decrease-and-conquer sorting.

Q1 Similar to what? / Principle	Principle: keep the left part sorted. Take the first item from the unsorted part and insert it into the correct position in the sorted part.
Q2 Time complexity	Worst/average O(n^2), best O(n) on nearly/already sorted input. Space O(1).
Q3 Stable?	Stable if insertion shifts only larger elements right and stops before equal elements.
Q4 Recursive vs iterative	Iteration inserts arr[i] into arr[0..i-1]. Recursion first sorts the prefix of length n-1, then inserts the last element.
Q5 Recursive disadvantage	Recursive insertion sort uses O(n) stack space. The insertion work is still linear per step, so total time remains O(n^2).

Iterative pseudocode

```
Procedure InsertionSort(arr)
  for i <- 1 to arr.length - 1 do
    key <- arr[i]
    j <- i - 1
    while j >= 0 and arr[j] > key do
      arr[j+1] <- arr[j]
      j <- j - 1
    arr[j+1] <- key
```

Recursive pseudocode

```
Procedure InsertionSort(arr)
  InsertSortPrefix(arr, arr.length)

Procedure InsertSortPrefix(arr, n)
  if n <= 1 then return
  InsertSortPrefix(arr, n - 1)
  key <- arr[n - 1]
  j <- n - 2
  while j >= 0 and arr[j] > key do
    arr[j+1] <- arr[j]
    j <- j - 1
  arr[j+1] <- key
```

6. Merge Sort

Week 6: divide-and-conquer faster sorting.

Q1 Similar to what? / Principle	Principle: divide array into two halves, recursively sort both halves, then merge two sorted halves.
Q2 Time complexity	O(n log n) time in best/average/worst cases. Extra space O(n) for merging.
Q3 Stable?	Stable if merge chooses the left item first when two keys are equal.
Q4 Recursive vs iterative	Natural version is recursive. Iterative bottom-up merge sort repeatedly merges blocks of size 1, 2, 4, 8, ...
Q5 Recursive disadvantage	Recursive version uses O(log n) call stack plus O(n) merge storage. It may be less memory-friendly than in-place simple sorts.

Iterative pseudocode

```
Procedure MergeSortIterative(arr)
  width <- 1
  while width < arr.length do
    left <- 0
    while left < arr.length do
      mid <- min(left + width, arr.length)
      right <- min(left + 2*width, arr.length)
      Merge(arr, left, mid, right)
      left <- left + 2*width
    width <- 2 * width
```

Recursive pseudocode

```
Procedure MergeSort(arr, left, right)
  if right - left <= 1 then return
  mid <- (left + right) / 2
  MergeSort(arr, left, mid)
  MergeSort(arr, mid, right)
  Merge(arr, left, mid, right)
```

7. Quick Sort

Week 6: divide-and-conquer faster sorting.

Q1 Similar to what? / Principle	Principle: choose a pivot, partition smaller items to the left and larger items to the right, then recursively sort both sides.
Q2 Time complexity	Average O(n log n), worst O(n^2) with bad pivots; space average O(log n) call stack, worst O(n).
Q3 Stable?	Not stable in the usual in-place implementation because partition swaps can change order of equal keys.
Q4 Recursive vs iterative	Natural version is recursive. Iterative version uses an explicit stack of subarray ranges.
Q5 Recursive disadvantage	Recursive quicksort can degrade to deep recursion O(n) with bad pivots. It also has worst-case O(n^2) time unless pivot choice is controlled.

Iterative pseudocode

```
Procedure QuickSortIterative(arr)
  stack.push((0, arr.length - 1))
  while stack not empty do
    (lo, hi) <- stack.pop()
```

```
    if lo < hi then
      p <- Partition(arr, lo, hi)
      stack.push((lo, p - 1))
      stack.push((p + 1, hi))
```

Recursive pseudocode

```
  Procedure QuickSort(arr, lo, hi)
    if lo >= hi then return
    p <- Partition(arr, lo, hi)
    QuickSort(arr, lo, p - 1)
    QuickSort(arr, p + 1, hi)
```

8. Counting Sort

Week 6: structured-input sorting.

Q1 Similar to what? / Principle	Principle: when keys are small integers in a known range, count how many times each key appears, then rebuild the sorted output.
Q2 Time complexity	O(n + k), where n is number of items and k is key range size. Space O(n + k) for stable version.
Q3 Stable?	Stable if using cumulative counts and filling the output array from right to left.
Q4 Recursive vs iterative	Normally iterative. A recursive version can recursively process key values or positions, but it is artificial.
Q5 Recursive disadvantage	Recursion adds stack overhead and does not improve the O(n+k) bound. Counting sort is naturally loop-based.

Iterative pseudocode

```
  Procedure CountingSort(arr, k)
    count[0..k] <- 0
    for x in arr do count[x] <- count[x] + 1
    index <- 0
    for value <- 0 to k do
      repeat count[value] times do
        arr[index] <- value
        index <- index + 1
```

Recursive pseudocode

```
  Procedure OutputCounts(value, k, count, arr, index)
    if value > k then return
    if count[value] > 0 then
      arr[index] <- value
      count[value] <- count[value] - 1
      OutputCounts(value, k, count, arr, index + 1)
    else
      OutputCounts(value + 1, k, count, arr, index)
```

9. Radix Sort

Week 6: structured-input sorting.

Q1 Similar to what? / Principle	Principle: sort numbers/strings digit by digit using a stable subroutine such as counting sort. Usually process least significant digit to most significant digit.

Q2 Time complexity	O(d(n+k)), where d is number of digits and k is base/range per digit. Space depends on the stable subroutine, usually O(n+k).
Q3 Stable?	Stable if every digit pass is stable. Stability is required for LSD radix sort to work correctly.
Q4 Recursive vs iterative	Normally iterative over digit positions. Recursive version can recurse over the digit index.
Q5 Recursive disadvantage	Recursive version adds stack depth O(d) and gives no main advantage over the loop over digits.

Iterative pseudocode

```
Procedure RadixSort(arr, d)
  for pos <- leastSignificant to mostSignificant do
    StableCountingSortByDigit(arr, pos)
```

Recursive pseudocode

```
Procedure RadixSortRec(arr, pos)
  if pos > mostSignificant then return
  StableCountingSortByDigit(arr, pos)
  RadixSortRec(arr, nextMoreSignificant(pos))
```

10. Quick Find (Union-Find)

Lecture 8a: Union-Find / Dynamic Connectivity.

Q1 Similar to what? / Principle	Principle: store a component id for every item. isSameGroup is quick because it only compares ids; union is slow because many ids may need changing.
Q2 Time complexity	isSameGroup O(1). union O(n). For many operations, this can be too slow.
Q3 Stable?	Not a sorting algorithm; stability is not applicable.
Q4 Recursive vs iterative	Iterative union scans the whole id array. Recursive scan can replace the loop.
Q5 Recursive disadvantage	Recursive union still scans n items and uses stack space if written recursively; iteration is clearer.

Iterative pseudocode

```
Procedure Union(p, q)
  pid <- id[p]; qid <- id[q]
  for i <- 0 to id.length - 1 do
    if id[i] = pid then id[i] <- qid

Procedure IsSameGroup(p, q)
  return id[p] = id[q]
```

Recursive pseudocode

```
Procedure UnionRec(p, q)
  ReplaceId(0, id[p], id[q])

Procedure ReplaceId(i, oldId, newId)
  if i = id.length then return
  if id[i] = oldId then id[i] <- newId
  ReplaceId(i + 1, oldId, newId)
```

11. Quick Union / Weighted Quick Union

Lecture 8a: Union-Find / Disjoint Set.

Q1 Similar to what? / Principle	Principle: represent each component as a tree. find follows parent links to the root. Weighted union attaches the smaller tree under the larger tree.
Q2 Time complexity	Quick Union find/union can be O(n) if the tree becomes tall. Weighted Quick Union is O(log n) without path compression; near constant amortized with path compression.
Q3 Stable?	Not a sorting algorithm; stability is not applicable.
Q4 Recursive vs iterative	find can be iterative by walking parent links or recursive by calling find(parent[x]).
Q5 Recursive disadvantage	Recursive find can be elegant, especially with path compression, but deep trees can cause stack overflow if not weighted/compressed.

Iterative pseudocode

```
Procedure Find(x)
  while parent[x] != x do
    x <- parent[x]
  return x

Procedure Union(p, q)
  rootP <- Find(p); rootQ <- Find(q)
  if rootP = rootQ then return
  if size[rootP] < size[rootQ] then
    parent[rootP] <- rootQ; size[rootQ] += size[rootP]
  else
    parent[rootQ] <- rootP; size[rootP] += size[rootQ]
```

Recursive pseudocode

```
Procedure Find(x)
  if parent[x] = x then return x
  parent[x] <- Find(parent[x])   // path compression
  return parent[x]

Procedure Union(p, q)
  rootP <- Find(p); rootQ <- Find(q)
  attach smaller root under larger root
```

12. Binary Heap MinPQ: add / delMin

Lecture 8b: priority queue implemented by binary heap.

Q1 Similar to what? / Principle	Principle: complete binary tree stored in an array. Heap property: every parent is <= its children. add uses swim; delMin replaces root by last item then sink.
Q2 Time complexity	add O(log n), delMin O(log n), getMin O(1), space O(n).
Q3 Stable?	Not a sorting algorithm. If used for heapsort, standard heapsort is not stable.
Q4 Recursive vs iterative	Heap operations are naturally iterative while moving up/down the tree. Recursive swim/sink is possible.
Q5 Recursive disadvantage	Recursive swim/sink uses call stack O(log n); iterative version is usually more direct.

Iterative pseudocode

```
Procedure Add(item)
  size <- size + 1
  heap[size] <- item
  Swim(size)

Procedure Swim(k)
  while k > 1 and heap[k] < heap[k/2] do
    swap(heap[k], heap[k/2])
    k <- k / 2

Procedure DelMin()
  min <- heap[1]
  swap(heap[1], heap[size])
  size <- size - 1
  Sink(1)
  return min
```

Recursive pseudocode

```
Procedure SwimRec(k)
  if k <= 1 then return
  if heap[k] < heap[k/2] then
    swap(heap[k], heap[k/2])
    SwimRec(k/2)

Procedure SinkRec(k)
  child <- smaller child of k
  if no child or heap[k] <= heap[child] then return
  swap(heap[k], heap[child])
  SinkRec(child)
```

13. Tree Traversals: Preorder / Inorder / Postorder / Level Order

Lecture 9: trees and tree traversal.

Q1 Similar to what? / Principle	Principle: visit all tree nodes in a chosen order. Preorder visits node before children; inorder visits left-node-right; postorder visits children before node; level order visits by breadth using a queue.
Q2 Time complexity	O(n) time because each node is visited once. Recursive DFS traversals use O(h) stack where h is tree height. Level order uses O(w) queue where w is max width.
Q3 Stable?	Not a sorting algorithm. Inorder traversal of a BST outputs keys in sorted order.
Q4 Recursive vs iterative	DFS traversals are naturally recursive. Iterative versions use a stack; level order uses a queue.
Q5 Recursive disadvantage	Recursive DFS can overflow on a very deep tree. Iterative stack/queue gives more explicit memory control.

Iterative pseudocode

```
Procedure InorderIter(root)
  stack <- empty
  current <- root
  while current != null or stack not empty do
    while current != null do
      stack.push(current)
      current <- current.left
    current <- stack.pop()
```

```
        visit(current)
        current <- current.right

    Procedure LevelOrder(root)
      queue.enqueue(root)
      while queue not empty do
        x <- queue.dequeue(); visit(x)
        enqueue non-null children of x
```

Recursive pseudocode

```
    Procedure Preorder(x)
      if x = null then return
      visit(x); Preorder(x.left); Preorder(x.right)

    Procedure Inorder(x)
      if x = null then return
      Inorder(x.left); visit(x); Inorder(x.right)

    Procedure Postorder(x)
      if x = null then return
      Postorder(x.left); Postorder(x.right); visit(x)
```

14. Depth First Search (DFS)

Lectures 9 and 11: graph traversal and graph paths.

Q1 Similar to what? / Principle	Principle: go as deep as possible before backtracking. Used for reachability, paths, connected components, topological order, and SCC algorithms.
Q2 Time complexity	O(V+E) with adjacency lists. Recursive stack O(V) in worst case; marked[] and edgeTo[] O(V).
Q3 Stable?	Not a sorting algorithm; stability is not applicable.
Q4 Recursive vs iterative	Recursive DFS follows the definition directly. Iterative DFS replaces recursion with an explicit stack.
Q5 Recursive disadvantage	Recursive DFS may overflow on large/deep graphs. Iterative DFS is safer for very large inputs.

Iterative pseudocode

```
    Procedure DFSIter(G, s)
      stack.push(s); marked[s] <- true
      while stack not empty do
        v <- stack.pop()
        for w in G.adj(v) do
          if not marked[w] then
            marked[w] <- true
            edgeTo[w] <- v
            stack.push(w)
```

Recursive pseudocode

```
    Procedure DFS(G, v)
      marked[v] <- true
      for w in G.adj(v) do
        if not marked[w] then
          edgeTo[w] <- v
          DFS(G, w)
```

15. Breadth First Search (BFS)

Lectures 9 and 11: graph traversal and shortest paths in unweighted graphs.

Q1 Similar to what? / Principle	Principle: visit vertices level by level using a queue. In an unweighted graph, BFS gives shortest paths by number of edges.
Q2 Time complexity	O(V+E) with adjacency lists. Space O(V) for queue, marked, edgeTo.
Q3 Stable?	Not a sorting algorithm; stability is not applicable.
Q4 Recursive vs iterative	BFS is naturally iterative with a queue. A recursive level-by-level version is possible but uncommon.
Q5 Recursive disadvantage	Recursive BFS is awkward because the queue/level frontier still has to be maintained; it is less clear than the iterative version.

Iterative pseudocode

```
Procedure BFS(G, s)
  queue.enqueue(s)
  marked[s] <- true
  while queue not empty do
    v <- queue.dequeue()
    for w in G.adj(v) do
      if not marked[w] then
        marked[w] <- true
        edgeTo[w] <- v
        queue.enqueue(w)
```

Recursive pseudocode

```
Procedure BFSRec(G, queue)
  if queue is empty then return
  v <- queue.dequeue()
  for w in G.adj(v) do
    if not marked[w] then
      marked[w] <- true
      edgeTo[w] <- v
      queue.enqueue(w)
  BFSRec(G, queue)
```

16. Dijkstra Shortest Paths

Lecture 10: weighted single-source shortest paths.

Q1 Similar to what? / Principle	Principle: repeatedly choose the unvisited vertex with smallest known distance, then relax its outgoing edges. Requires non-negative edge weights.
Q2 Time complexity	With indexed binary heap: O((V+E) log V). With simple array priority selection: O(V^2). Space O(V+E).
Q3 Stable?	Not a sorting algorithm; stability is not applicable.
Q4 Recursive vs iterative	Normally iterative using a priority queue. Recursive version can recurse after processing one min vertex, but it is artificial.
Q5 Recursive disadvantage	Recursive wrapper does not improve complexity and may create unnecessary stack depth O(V). The priority queue loop is clearer.

Iterative pseudocode

```
Procedure Dijkstra(G, s)
  for each v: dist[v] <- infinity
  dist[s] <- 0
  pq.add(s, 0)
  while pq not empty do
    v <- pq.delMin()
    for edge v->w with weight c do
      if dist[w] > dist[v] + c then
        dist[w] <- dist[v] + c
        edgeTo[w] <- v
        pq.decreaseKey(w, dist[w])
```

Recursive pseudocode

```
Procedure DijkstraRec(G, pq)
  if pq is empty then return
  v <- pq.delMin()
  for each edge v->w do Relax(v, w)
  DijkstraRec(G, pq)
```

17. Prim Minimum Spanning Tree

Lecture 10: MST and cut property.

Q1 Similar to what? / Principle	Principle: grow one tree. Repeatedly add the minimum-weight edge that connects the current tree to a new vertex.
Q2 Time complexity	Lazy Prim often O(E log E). Eager Prim with indexed PQ O(E log V). Space O(V+E).
Q3 Stable?	Not a sorting algorithm; stability is not applicable.
Q4 Recursive vs iterative	Usually iterative with a priority queue. Recursive version can repeatedly add the next lightest crossing edge.
Q5 Recursive disadvantage	Recursive style adds stack overhead and hides the priority queue process. Iteration matches the algorithm better.

Iterative pseudocode

```
Procedure Prim(G, s)
  visit(s)
  while pq not empty and MST has fewer than V-1 edges do
    e <- pq.delMin()
    if both endpoints of e are marked then continue
    add e to MST
    visit(the unmarked endpoint)

Procedure visit(v)
  marked[v] <- true
  for each edge e from v do
    if other endpoint not marked then pq.add(e)
```

Recursive pseudocode

```
Procedure PrimRec(G, pq)
  if pq empty or MST complete then return
  e <- pq.delMin()
  if e connects to an unmarked vertex then
    add e to MST
    visit(unmarked endpoint)
  PrimRec(G, pq)
```

18. Kruskal Minimum Spanning Tree

Lecture 10: MST and cut property.

Q1 Similar to what? / Principle	Principle: sort edges by weight. Scan from smallest to largest; add an edge if it connects two different components. Union-Find detects cycles.
Q2 Time complexity	Sorting edges costs O(E log E). Union-Find operations are near constant amortized with path compression. Overall O(E log E).
Q3 Stable?	Not a sorting algorithm itself, but it depends on sorting edges. Stability of edge sort does not affect MST weight.
Q4 Recursive vs iterative	Iterative scan over sorted edges is natural. Recursive version can process one edge index at a time.
Q5 Recursive disadvantage	Recursive scan may use O(E) stack space and gives no improvement over the loop.

Iterative pseudocode

```
Procedure Kruskal(G)
  sort all edges by weight
  uf <- new UnionFind(V)
  for e in sorted edges do
    v,w <- endpoints of e
    if not uf.connected(v,w) then
      uf.union(v,w)
      add e to MST
    if MST has V-1 edges then break
```

Recursive pseudocode

```
Procedure KruskalRec(edges, i)
  if MST complete or i = edges.length then return
  e <- edges[i]
  if endpoints not connected then
    union endpoints; add e
  KruskalRec(edges, i + 1)
```

19. Connected Components by DFS

Lecture 11: connectivity queries in undirected graphs.

Q1 Similar to what? / Principle	Principle: run DFS from every unmarked vertex. Each DFS marks one connected component with the same component id.
Q2 Time complexity	Preprocessing O(V+E). Query connected(v,w) is O(1) by comparing component ids. Space O(V).
Q3 Stable?	Not a sorting algorithm; stability is not applicable.
Q4 Recursive vs iterative	DFS part is naturally recursive; the outer loop over vertices is iterative. Fully recursive outer scan is possible.
Q5 Recursive disadvantage	Recursive DFS can overflow on large components. The recursive outer loop is less readable than a simple for loop.

Iterative pseudocode

```
Procedure Components(G)
  count <- 0
  for v <- 0 to V-1 do
    if not marked[v] then
      DFSMark(G, v, count)
      count <- count + 1
```

```
Procedure Connected(v,w)
  return id[v] = id[w]
```

Recursive pseudocode

```
Procedure DFSMark(G, v, componentId)
  marked[v] <- true
  id[v] <- componentId
  for w in G.adj(v) do
    if not marked[w] then DFSMark(G, w, componentId)
```

20. Topological Sort

Lecture 11: DAG ordering by DFS reverse postorder.

Q1 Similar to what? / Principle	Principle: for a directed acyclic graph, run DFS and put each vertex onto a list after all descendants are processed. Reverse postorder gives a valid topological order.
Q2 Time complexity	O(V+E) time and O(V) space. Only valid for DAGs.
Q3 Stable?	Not a sorting-by-key algorithm; stability is not applicable. Multiple valid topological orders may exist.
Q4 Recursive vs iterative	Recursive DFS is the standard version. Iterative version needs a stack with state to simulate postorder.
Q5 Recursive disadvantage	Recursive version may overflow on very deep DAGs. Also, if graph has a cycle, topological order is invalid unless cycle detection is added.

Iterative pseudocode

```
Procedure TopologicalSortIter(G)
  for each unmarked vertex s do
    simulate DFS with stack of (vertex, next-neighbor-index)
    when a vertex finishes, push it to postorder
  return reverse(postorder)
```

Recursive pseudocode

```
Procedure DFS(v)
  marked[v] <- true
  for w in G.adj(v) do
    if not marked[w] then DFS(w)
  postorder.add(v)

Procedure TopologicalSort(G)
  run DFS from all unmarked vertices
  return reverse(postorder)
```

21. Kosaraju Strongly Connected Components

Lecture 11: SCCs in directed graphs.

Q1 Similar to what? / Principle	Principle: reverse the graph, compute reverse postorder by DFS, then run DFS on the original graph in that order. Each DFS gives one strongly connected component.
Q2 Time complexity	O(V+E) time and O(V+E) space for graph/reverse graph plus arrays.
Q3 Stable?	Not a sorting algorithm; stability is not applicable.
Q4 Recursive vs iterative	The two DFS phases are naturally recursive; iterative DFS can

	be used for both phases.
Q5 Recursive disadvantage	Recursive DFS may overflow on large graphs. The algorithm is conceptually tricky because order from the reversed graph is essential.

Iterative pseudocode

```
Procedure Kosaraju(G)
  order <- ReversePostorder(Reverse(G))
  count <- 0
  for v in order do
    if not marked[v] then
      DFSAssign(G, v, count)
      count <- count + 1
```

Recursive pseudocode

```
Procedure DFSAssign(G, v, componentId)
  marked[v] <- true
  id[v] <- componentId
  for w in G.adj(v) do
    if not marked[w] then DFSAssign(G, w, componentId)
```

22. Binary Search Tree: get / put

Lecture 12: maps, symbol tables, BST.

Q1 Similar to what? / Principle	Principle: for each node, left subtree keys are smaller and right subtree keys are larger. Search/insert follows comparisons from root to a null link.
Q2 Time complexity	O(h), where h is tree height. Balanced tree: O(log n). Worst unbalanced chain: O(n). Space recursive O(h), iterative O(1).
Q3 Stable?	Not a sorting algorithm. Inorder traversal of BST is sorted by key.
Q4 Recursive vs iterative	BST get/put can be written iteratively by walking down the tree or recursively by returning updated subtrees.
Q5 Recursive disadvantage	Recursive version is elegant but uses O(h) stack; if the tree degenerates to a chain, h can be n.

Iterative pseudocode

```
Procedure Get(root, key)
  x <- root
  while x != null do
    if key = x.key then return x.val
    else if key < x.key then x <- x.left
    else x <- x.right
  return null

Procedure Put(root, key, val)
  walk down until null position, then attach new node
```

Recursive pseudocode

```
Procedure Get(x, key)
  if x = null then return null
  if key = x.key then return x.val
  if key < x.key then return Get(x.left, key)
  else return Get(x.right, key)

Procedure Put(x, key, val)
```

```
if x = null then return new Node(key,val)
if key < x.key then x.left <- Put(x.left,key,val)
else if key > x.key then x.right <- Put(x.right,key,val)
else x.val <- val
return x
```

23. 2-3 Trees / Red-Black Trees conceptual operations

Lecture 12: balanced search trees.

Q1 Similar to what? / Principle	Principle: keep the search tree balanced. A 2-3 tree allows 2-nodes and 3-nodes. A left-leaning red-black tree represents 3-nodes using red links and fixes balance using rotations and color flips.
Q2 Time complexity	Search/insert O(log n) because height remains logarithmic. Rotations and color flips are O(1) local operations.
Q3 Stable?	Not a sorting algorithm; stability is not applicable.
Q4 Recursive vs iterative	Insertion is usually recursive: insert like BST, then fix red-black violations on the way back. Iterative implementation is possible but more complex.
Q5 Recursive disadvantage	Recursive balancing logic is harder to trace and debug. Many small cases: rotate left, rotate right, flip colors.

Iterative pseudocode

```
Procedure PutIterLLRB(root, key, val)
  insert as in BST while tracking path
  while moving back up the path:
    if right red and left not red: rotateLeft
    if left red and left.left red: rotateRight
    if both children red: flipColors
  root.color <- BLACK
```

Recursive pseudocode

```
Procedure Put(h, key, val)
  if h = null then return new RED node
  if key < h.key then h.left <- Put(h.left,key,val)
  else if key > h.key then h.right <- Put(h.right,key,val)
  else h.val <- val
  if isRed(h.right) and not isRed(h.left) then h <- rotateLeft(h)
  if isRed(h.left) and isRed(h.left.left) then h <- rotateRight(h)
  if isRed(h.left) and isRed(h.right) then flipColors(h)
  return h
```

24. Hash Table with Separate Chaining

Lecture 13: hashing, collision handling, resizing.

Q1 Similar to what? / Principle	Principle: compute hash(key) to choose an array index. If multiple keys collide at the same index, store them in a linked list or small collection at that bucket.
Q2 Time complexity	Average/amortized O(1) for add/contains/get if hash function distributes well and resizing controls load factor. Worst O(n) if all keys collide.
Q3 Stable?	Not a sorting algorithm; stability/order is generally not guaranteed.
Q4 Recursive vs iterative	Normally iterative: compute bucket, scan chain, add/update.

	Recursive chain search is possible.
Q5 Recursive disadvantage	Recursive search through a long chain can use O(chain length) stack and gives no advantage over a loop.

Iterative pseudocode

```
Procedure Contains(key)
  i <- hash(key) mod table.length
  for each node in table[i] do
    if node.key = key then return true
  return false

Procedure Add(key)
  if load factor too high then resize
  i <- hash(key) mod table.length
  if key not already in table[i] then add key to chain
```

Recursive pseudocode

```
Procedure ContainsNode(node, key)
  if node = null then return false
  if node.key = key then return true
  return ContainsNode(node.next, key)

Procedure Contains(key)
  i <- hash(key) mod table.length
  return ContainsNode(table[i], key)
```

Source notes

- Week 4 introduced recursion, recursive linear search, and comparison between recursive and iterative solutions.
- Week 5 introduced linear search, binary search, bubble sort, selection sort, insertion sort, and complexity analysis.
- Week 6 introduced merge sort, quick sort, counting sort, radix sort, and divide-and-conquer complexity.
- Lecture 8a introduced Union-Find / Disjoint Set for dynamic connectivity.
- Lecture 8b introduced Priority Queue and Binary Heap, including swim and sink.
- Lecture 9 introduced tree traversals, graphs, DFS, BFS, and graph representations.
- Lecture 10 introduced Dijkstra shortest paths, Prim MST, and Kruskal MST.
- Lecture 11 introduced connected components, topological sort, and Kosaraju SCC.
- Lecture 12 introduced Map, BST get/put, 2-3 trees, and red-black trees.
- Lecture 13 introduced hash tables, hashing, collision handling with separate chaining, resizing, and hash-code distribution.